DevOps Infrastructure – Unit 1 Notes (Part 2)

Container Runtime, Process Isolation & Namespaces

---

1. Container Runtime

Definition:

A **container runtime** is the software responsible for:

- Running containers
- Managing container lifecycle (create, start, stop, delete)

It acts as a bridge between:

- The **container image**
- The **host operating system**

Types of Container Runtimes

a) Low-Level Runtimes

These directly interact with the OS kernel.

Examples:

- **runc** (used by Docker)
- containerd (core runtime engine)

b) High-Level Runtimes

Provide user-friendly interfaces and additional features.

Examples:

- Docker
- CRI-O (used in Kubernetes)

Responsibilities of a Container Runtime

- Pull container images
- Create container environment
- Manage execution of processes inside containers

- Handle resource allocation
- Maintain isolation between containers

How It Works (Simplified Flow)

1. User runs a command (e.g., docker run)
2. Runtime pulls image from registry
3. Sets up isolated environment
4. Starts container process

---

2. Process Isolation in Containers

What is Process Isolation?

Process isolation ensures that:

- Each container runs independently
- Processes inside one container cannot interfere with others

---

Why It is Important

- Security: prevents unauthorized access
- Stability: failure in one container doesn't affect others
- Multi-tenancy: multiple apps can run on same host safely

How Isolation is Achieved

Linux provides **kernel-level features**:

- Namespaces
- Control Groups (covered in next part)

---

3. Namespaces (Core Concept)

Definition:

Namespaces are a Linux kernel feature that isolates system resources for processes.

Each container gets its own **view of the system**.

Types of Namespaces

1. PID Namespace (Process ID)

- Each container has its own process tree
- Process IDs start from 1 inside container

Example:

- A process may be PID 1 inside container but PID 5000 on host

---

2. NET Namespace (Networking)

- Isolates network interfaces
- Each container gets:
  - Its own IP address
  - Its own network stack

---

3. MNT Namespace (Mount/File System)

- Isolates file system view
- Each container sees its own file system

---

4. UTS Namespace (Hostname)

- Allows each container to have its own hostname

---

5. IPC Namespace (Inter-Process Communication)

- Isolates shared memory and message queues

---

6. USER Namespace

- Maps user IDs inside container to different IDs on host
- Enhances security

Visualization

Think of namespaces like:

Each container is in its own “bubble”
→ It thinks it owns the system
→ But actually shares the host OS

---

4. Combined Working (Runtime + Namespaces)

When a container starts:

1. Runtime creates namespaces
2. Assigns isolated resources
3. Starts process inside that isolated environment

---

5. Key Advantages

- Strong isolation without heavy virtualization
- Efficient resource usage
- Better security compared to simple process execution

---

6. Summary

- Container runtime manages execution of containers
- Process isolation ensures independent execution
- Namespaces provide logical separation of system resources
- Containers rely on OS-level isolation instead of full OS virtualization

---